CS315 Internet Programming – Fall 2026

HW4 React Contact Directory

Total: 10 points

Weight: 10% of the Final Course Grade

Purpose of this assignment:

This assignment is a short React consolidation project. You will build a contact directory from a provided hard-coded array. The goal is to practice the React ideas we have already covered without introducing an external API or a backend.

Important: This assignment does not use fetch(), JSONPlaceholder, Flask, or a database. The contact records are already provided in the starter code.

Learning Goals

- Use a TypeScript interface to describe contact data.
- Use a reusable React component with typed props.
- Use map() to generate multiple components from an array.
- Use a unique React key for list items.
- Use useState() and an event handler to change the UI.
- Build a production version with Vite and **deploy it to ICE server**.

What You Are Building

Your application will contain **two** major sections:

- Featured Contact: **shows one contact at a time**. A **Next Contact** button moves to the next contact and **returns to the first contact after the last one**.
- All Contacts: shows every contact by using contacts.map(...) and a reusable <ContactCard /> component.

Conceptual flow:

```
Contact[]
   |
   +--> selectedIndex state --> Featured Contact
   |
   +--> map() --> ContactCard --> rendered directory
```

Contact[] is used in **two** ways in this assignment. First, **selectedIndex** identifies one contact from the array and displays that contact as the Featured Contact. Second, **map()** processes

every contact in the array and creates one ContactCard component for each record, producing the full rendered directory.

This is the whole Contact array:

```
const contacts: Contact[] = [
  ...
];
```

And the following is to use this:

```
const [selectedIndex, setSelectedIndex] = useState(0);
```

to memorize which record we had clicked?

For example, contacts[selectedIndex]

When we click the Next Contact button →

```
setSelectedIndex((selectedIndex + 1) % contacts.length);
```

Contact[]

↓

selectedIndex state

↓

Featured Contact

(This is to pick up 1 contact as a Featured Contact)

Second, this pipeline is **to process all the contacts**:

Contact[]

↓

map()

↓

ContactCard

↓

rendered directory

The following sample code is to generate a ContactCard for each of the contact

```
contacts.map(function(contact) {
  return (
    <ContactCard
      key={contact.id}
      name={contact.name}
      email={contact.email}
      department={contact.department}
      phone={contact.phone}
    />
  );
})
```

Starter Project

A starter ZIP is provided. It is already organized as a small React + TypeScript + Vite project. Do not upload node_modules.

Local setup

- Extract CS315_HW4_Student_Starter.zip.
- Open the extracted folder in VS Code.
- Choose Terminal -> New Terminal.
- Run npm.cmd install (in Windows Powershell) or npm install (in Windows Command Prompt).
- Start the development server with npm.cmd run dev.

The starter project includes these important files:

```
src/
  App.tsx
  ContactCard.tsx
  App.css
  index.css
  main.tsx

vite.config.ts
package.json
```

The provided contact records are already hard-coded in App.tsx. You should use those records. You may change names or departments for presentation purposes, but you may not replace the assignment with an external API.

Part 1 - Typed Contact Data (2 points)

The starter code already provides a Contact interface and a Contact[] array. You are responsible for understanding and preserving the typed structure.

```
interface Contact {
  id: number;
  name: string;
  email: string;
  department: string;
  phone: string;
}
const contacts: Contact[] = [ ... ];
```

- The array must contain at least the provided eight contacts.
- Each record must include id, name, email, department, and phone.
- Do not convert this assignment into a plain JavaScript project.

Part 2 - Reusable ContactCard Component (2 points)

Complete **ContactCard.tsx**. Define a typed props structure and render the four contact fields. You may use **props.name** style or **destructuring**.

```
type ContactCardProps = {
  name: string;
  email: string;
  department: string;
  phone: string;
};
```

Each contact card must visibly display name, department, email, and phone.

Part 3 - Dynamic Rendering with map() (2 points)

In App.tsx, use contacts.map(...) to generate one ContactCard for each contact.

- Use contacts.map(...).
- Use key={contact.id}.
- Pass name, email, department, and phone as props.
- Do not manually write eight separate <ContactCard /> elements.

The intended flow is:

```
Contact[]
   |
 map()
   |
one Contact object at a time
   |
<ContactCard ... />
   |
rendered UI
```

Part 4 - State and User Interaction (2 points)

Create a state variable named selectedIndex with an initial value of 0. Use it to control the Featured Contact section.

const [selectedIndex, setSelectedIndex] = useState(0);

Create a function named **showNextContact()**. Each click should move to the next contact. After the final contact, the index should return to 0.

Hint:

```
(selectedIndex + 1) % contacts.length
```

- Display the selected contact name and department in the Featured Contact section.
- Add a Next Contact button.
- The button must use a React event handler and update state through the setter function.
- The page should update **without** a browser **refresh**.
- showNextContact() makes the use of setSelectedIndex() function, and you need to figure out the parameter for setSelectedIndex() function ← I already tell you the answer!

Part 5 - Styling, Build, Deployment, and Evidence (2 points)

- Keep the starter layout readable and add at least two meaningful CSS improvements of your own.
- Build the production version locally with Vite.
- Deploy the production build to ~/public_html/assignment4/ on ICE.
- Submit a working ICE URL and screenshots as described later in this document.

ICE Server Deployment - Read This Carefully

Development and build happen on your local computer. ICE is used only for deployment. ***Do not install React, Vite, or npm packages on ICE for this assignment.***

Deployment overview

```
Local React / TypeScript source
        |
        | npm.cmd run build
        v
      dist/
        |
        | upload contents of dist/
        v
~/public_html/assignment4/
        |
        v
Public ICE website
```

Step 1 - Finish and test locally

npm.cmd run dev

Open the Vite local URL (usually similar to http://localhost:5173/). Before deployment, verify that the contact cards, Next Contact button, and styling work locally.

Step 2 - Keep the ICE-friendly Vite setting

The starter **vite.config.ts** already contains the required relative base path. Do not remove it:

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  base: './',
});
```

The line base: './' is important because your application will be hosted inside a subdirectory such as /~USERNAME/assignment4/, not at the root of the ICE web server.

Step 3 - Build the production version locally

```
npm.cmd run build
```

If the build succeeds, Vite creates a dist folder. The exact asset filenames may differ:

```
dist/
```

```
index.html
assets/
  index-xxxxx.js
  index-xxxxx.css
```

The dist folder is the production build. You will deploy the contents of this folder. Do not deploy src/ or node_modules/ as the public website.

Step 4 - Connect to ICE Server

If you are off campus, connect to the Truman VPN first when required. Then connect with SSH:

```
ssh USERNAME@ice.truman.edu
```

Replace USERNAME with your Truman username.

Step 5 - Create the deployment directory

```
cd ~
mkdir -p ~/public_html
mkdir -p ~/public_html/assignment4
```

Expected structure:

```
/home/ldap/USERNAME/
  public_html/
    assignment4/
```

Step 6 - Set directory permissions

chmod 711 ~
chmod 755 ~/public_html
chmod 755 ~/public_html/assignment4

Recommended permissions:

Location	Permission	Purpose
Home directory (~)	711	Owner has full access; others can traverse but cannot list the directory
~/public_html	755	Web server can traverse/read the public web directory
~/public_html/assignment4	755	Web server can traverse/read the assignment directory
Website directories	755	Directories need execute/traverse permission
Website files	644	Files are readable but not world-writable

Do not use chmod -R 777. Your web files do not need to be writable by everyone.

Step 7 - Upload the contents of dist/

Return to a terminal on your local Windows computer. Run the command from the React project folder:

```
scp -r .\dist\* USERNAME@ice.truman.edu:~/public_html/assignment4/
```

Replace USERNAME with your Truman username.

[Important] Upload the contents of dist/, not the dist folder itself.

Correct:

```
public_html/
  assignment4/
    index.html
    assets/
      index-xxxxx.js
      index-xxxxx.css
```

Incorrect:

```
public_html/
  assignment4/
    dist/
      index.html
      assets/
```

Step 8 - Normalize permissions after upload

Log in to ICE again and run:

```
find ~/public_html/assignment4 -type d -exec chmod 755 {} \;
find ~/public_html/assignment4 -type f -exec chmod 644 {} \;
```

Check the top level:

```
ls -l ~/public_html/assignment4
```

You should see index.html directly inside assignment4 and an assets directory.

Step 9 - Test the public URL

```
http://ice.truman.edu/~USERNAME/assignment4/
```

- The page loads from ICE, not just localhost.
- CSS is applied.
- All contact cards appear.
- The Next Contact button still works.
- There are no obvious 404 errors in the browser Developer Tools Console/Network tabs.

Step 10 - If you change the source after deployment

Changing App.tsx or ContactCard.tsx on your laptop does not automatically change the ICE website. Repeat the build-and-upload cycle:

```
modify source code
      |
npm.cmd run build
      |
upload the new dist/ contents
      |
test the ICE URL again
```

Submission Requirements

Submit both items below in Brightspace.

1. Source Code ZIP

Name it similar to: **Assignment4_LastName.zip**. Include the source/configuration files needed to review your work, such as:

```
src/
  App.tsx
  ContactCard.tsx
  App.css
  index.css
  main.tsx

index.html
package.json
vite.config.ts
tsconfig.json
tsconfig.app.json
tsconfig.node.json
```

Do not include node_modules/. It is large and can be **recreated** with **npm.cmd install** (Windows Powershell). **You also do not need to include dist/** in the Brightspace source ZIP.

2. Results PDF or DOCX

Name it similar to: **Assignment4_Results_LastName.pdf**. Include:

- Your full working ICE URL.
- Screenshot 1: the deployed directory showing multiple contact cards.
- Screenshot 2: the Featured Contact section before clicking Next Contact.
- Screenshot 3: the Featured Contact section after clicking Next Contact.
- A short sentence confirming that you tested the ICE URL after deployment.

Grading Summary

Part	Requirement	Points
1	Typed Contact data and Contact[] structure	2
2	Reusable ContactCard with typed props	2
3	Dynamic rendering with map() and key	2
4	selectedIndex state and Next Contact interaction	2
5	CSS, production build, ICE deployment, and evidence	2
	Total	**10**

Code Verification

You are responsible for understanding the code you submit. You may be asked to explain or make a small change to your own program. Be prepared to explain concepts such as Contact[], props, map(), key, useState(), the setter function, and the click handler.

Final Checklist

☐ The project runs locally with npm.cmd run dev.

☐ ContactCard.tsx uses typed props.

☐ All contacts are rendered using contacts.map(...).

☐ Each generated ContactCard has key={contact.id}.

☐ The Featured Contact is controlled by selectedIndex state.

☐ The Next Contact button changes the featured contact.

☐ I added at least two meaningful CSS improvements.

☐ vite.config.ts still contains base: './'.

☐ npm.cmd run build succeeds.

☐ The contents of dist/ were uploaded directly into ~/public_html/assignment4/.

☐ Home permission is appropriate (711) and web directories are traversable (755).

☐ Website files are readable (644).

☐ The public ICE URL works in a browser.

☐ I tested the deployed interaction, not only localhost.

☐ My Brightspace ZIP does not contain node_modules/.

☐ My results document contains the URL and required screenshots.

Remember: for this assignment, your laptop is the development/build machine, and ICE is the deployment server.